Platform Engineering & Kubernetes

Module 1: Platform Engineering & IDPs

Mojtaba Imani

June 2026

Agenda

1. The Developer vs. Operations Challenge
2. From DevOps to Platform Engineering
3. Internal Developer Platforms (IDP)
4. Platform Tools & CNCF Ecosystem
5. IDP Architecture Options
6. Platform vs. Developer Applications
7. GitOps & ArgoCD

The Dev vs. Ops Challenge

- **Developers**: Focus on writing features.
  - "It works on my machine!"
- **Operations**: Focus on stability, security, and uptime.
  - "But it doesn't work in production!"

The DevOps Solution

- "You build it, you run it."
- **Reality**: Overwhelmed developers who must master cloud providers, Kubernetes, CI/CD, and security, etc.

What is Platform Engineering?

Platform Engineering is the discipline of designing and building toolchains and workflows that enable **developer self-service** in the cloud-native era.

- **Goal**: Reduce cognitive load for developers.
- **Product Mindset**: The platform is a product; developers are the customers.
- **Platform Engineers**: Build and maintain the platform.

Career path: Developer/Platform Engineer/Infrastructure Engineer

Internal Developer Platform (IDP)

An IDP is the set of tools, services, and technologies integrated by the platform team to create a cohesive developer experience.

- **Platform Tools**: Pre-configured shared services (databases, CI/CD, monitoring).
- **Templates**: Scaffolding for new microservices and pipelines.
- **Self-Service**: Provisioning resources without operations tickets.
- **Golden Path**: The pre-paved, secure, and compliant default route.

Benefits of an IDP

- **For Developers**:
  - Focus on writing business logic, not YAML or infrastructure.
  - Faster time-to-market.
- **For Platform Teams**:
  - Shift from resolving repetitive tickets to building platform features.
  - Standardized environments and zero configuration drift.
- **For the Organization**:
  - Built-in security and compliance by default.
  - Unified way of working across all teams.

Common Challenges Without a Platform

- Manual Kubernetes setup leads to technical debt and scaling issues.
- Managing multiple environments is inconsistent.
- Different teams use varied tools and workflows.
- Lack of self-service overwhelms IT with developer requests.
- Developer-created infrastructure can expose security vulnerabilities.

IDP Categories & CNCF Ecosystem

- **Infrastructure as Code**: Terraform, OpenTofu, Crossplane.
- **Observability**: Prometheus, Grafana, Loki, Jaeger, OpenTelemetry.
- **Application Packaging**: Helm, Kustomize.
- **Ingress & SSL**: Ingress-Nginx, Traefik, Cert-Manager, Gateway API.
- **Security & Policy**: RBAC, Kyverno, Keycloak.
- **GitOps**: ArgoCD, FluxCD.
- **Secret Management**: HashiCorp Vault, External-Secrets Operator.
- **Developer Portals**: Backstage.

IDP Architecture: Cloud vs. Kubernetes

Option 1: Cloud-Specific Services

- **Pros**: Reduced operational overhead, high availability, built-in security.
- **Cons**: Vendor lock-in, higher costs, limited customization.

Option 2: Inside Kubernetes Cluster

- **Pros**: Complete control, cloud-agnostic, open-source ecosystem.
- **Cons**: High complexity, requires deep Kubernetes expertise.

Platform vs. Developer Applications

Two categories of deployment on the platform:

- **Platform tools**: Shared services (ArgoCD, Prometheus, Keycloak) managed by the platform team.
- **Developer applications**: Business microservices deployed by development teams.

Kubernetes Platform Architecture

- Mixed or separate clusters per environment (Dev, Test, Staging, Production).
- Mixed or separate clusters for platform tools vs. developer apps.

Introduction to GitOps

GitOps is an operational framework that uses Git as the single source of truth for infrastructure and application declarations.

1. **Declarative**: Entire system described in code.
2. **Version Controlled**: Desired state stored in Git.
3. **Automated Delivery**: Changes merged to Git are applied automatically.
4. **Self-Healing**: Automated agents reconcile cluster drift.

Audit Trail:
Track who, when, what, why, and who approved each change. Roll back to the last known working version on failure.

GitOps with ArgoCD

ArgoCD is a declarative, GitOps continuous delivery tool for Kubernetes.

- **Auto-Syncing**: Automatically pulls changes from Git and applies them.
- **Auto-Pruning**: Deletes resources in the cluster that are removed from Git.
- **Self-Healing**: Reverts manual cluster changes back to the Git state.

GitOps Best Practices

- **DRY Approach**: Same templates for all environments, customized using environment values.
- **ArgoCD App of Apps**: A single root application manages the entire platform.
- **Pre-Integrated Dashboards**: ArgoCD, Grafana, Keycloak.
- **Secrets never in Git**: Store credentials in Vault or a secrets operator; reference them declaratively in manifests.

Platform Installation Steps

1. Provision Kubernetes cluster using IaC tools (Terraform) or on-premises tools.
2. Install and configure platform tools and microservices using GitOps (ArgoCD or Flux)